

расстоянии до/последнего; при передвижении второго указателя на следующую страницу, для нее очищается признак использования. Таким образом, интервал времени, за который страница должна быть использована, чтобы ее не вытеснили, уменьшается, и в памяти остаются только действительно часто используемые страницы.

3. ВЗАИМОДЕЙСТВИЕ ПОТОКОВ – ПЕРЕДАЧА ДАННЫХ И СИНХРОНИЗАЦИЯ

Даже в однопроцессорной многозадачной системе процессы чередуются (разделяют процессор), создавая иллюзию параллельного выполнения. В многопроцессорной системе наличие нескольких параллельно выполняющихся процессов (или потоков), работающих под управлением ОС, является необходимым условием ее функционирования.

Таким образом, концепция параллельных вычислений является одной из ключевых концепций построения современных ОС. Параллельность охватывает множество вопросов разработки, включая вопросы обмена информацией между процессами, деления ресурсов, синхронизацию работы процессов.

Потоки могут выполняться параллельно (независимо друг от друга, в случае многопроцессорной системы) до тех пор, пока не возникнет потребность в общении между ними. Для обеспечения совместной работы потоков необходимо предпринимать усилия – специальным образом планировать и управлять ими. Во многих ОС присутствуют специальные средства, обеспечивающие их синхронизацию и общение и устанавливающие определенные ограничения на последовательность выполнения взаимосвязанных параллельных потоков. Реализация синхронизирующих правил осуществляется с помощью механизмов синхронизации. Такие механизмы весьма многочисленны по способам реализации, отличаются степенью эффективности областями использования в различных приложениях.

Особенности каждого конкретного типа взаимодействия между двумя или более параллельными потоками определяются задачей синхронизации. Количество различных задач синхронизации неограниченно. Однако некоторые из них являются типичными. К ним относятся: взаимное исключение, «производители – потребители», «читатели – писатели», «обедающие философы» и т.д. Большинство задач в реальных ОС по согласованию параллельных потоков можно решить либо с помощью этих типовых задач, либо с помощью их модификаций.

3.1. Взаимодействие потоков

Если приложение реализовано в виде множества процессов (или потоков), то эти процессы (потоки) могут взаимодействовать тремя основными способами:

- 1) посредством разделения памяти (оперативной или внешней);
- 2) посредством передачи сообщений или потоков данных;
- 3) посредством использования одних и тех же объектов операционной системы.

При взаимодействии через общую память процессы должны синхронизовать свое выполнение. Пренебрежение вопросами синхронизации в многопоточной системе может привести к неправильной работе приложения или даже к его краху.

Пример. Задача ведения базы данных клиентов некоторого предприятия.

Каждому клиенту отводится отдельная запись в базе данных, в которой среди прочих полей имеются поля Заказ и Оплата. Программа, ведущая базу данных, оформлена как единый процесс, имеющий несколько потоков, в том числе поток А (писатель 1), который заносит в базу данных информацию о заказах, поступивших от клиентов, и поток В (писатель 2), который фиксирует в базе данных сведения об оплате клиентами выставленных счетов. Оба эти потока совместно работают над общим файлом базы данных, используя однотипные алгоритмы, включающие три шага:

1. Считать из файла базы данных в буфер запись о клиенте с заданным идентификатором.
2. Внести новое значение в поле Заказ (для потока А) или Оплата (для потока В).
3. Вернуть модифицированную запись в файл базы данных.

Обозначим соответствующие шаги для потока А как А1, А2 и А3, а для потока В как В1, В2 и В3. Предположим, что в некоторый момент поток А обновляет поле Заказ записи о клиенте N. Для этого он считывает эту запись в свой буфер (шаг А1), модифицирует значение поля Заказ (шаг А2), но внести запись в базу данных (шаг А3) не успевает, так как его выполнение прерывается, например, вследствие завершения кванта времени.

Предположим также, что потоку В также потребовалось внести сведения об оплате относительно того же клиента N. Когда подходит очередь потока В, он успевает считать запись в свой буфер (шаг В1)

и выполнить обновление поля Оплата (шаг В2), а затем прерывается. Заметим, что в буфере у потока В находится запись о клиенте N, в которой поле Заказ имеет прежнее, неизмененное значение.

Когда в очередной раз управление будет передано потоку А, то он, продолжая свою работу, сделает запись о клиенте N с модифицированным полем Заказ в базу данных (шаг А3). После прерывания потока А и активизации потока В последний запишет в базу данных поверх только что обновленной записи о клиенте N свой вариант записи, в которой обновлено значение поля Оплата. Таким образом, в базе данных будут зафиксированы сведения о том, что клиент N произвел оплату, но информация о его заказе окажется потерянной (рис. 36).

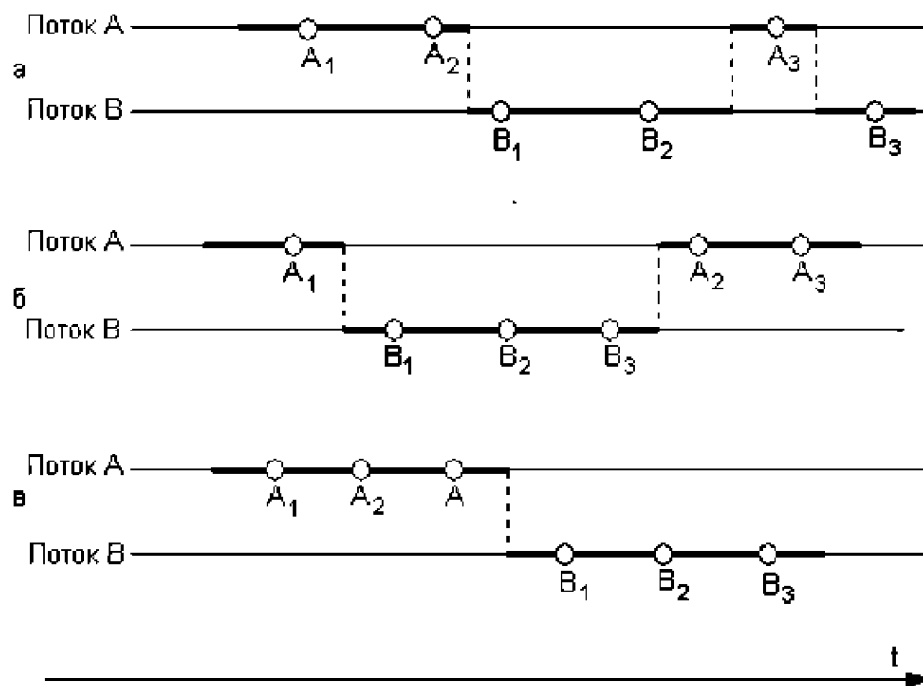


Рис. 36. Влияние относительных скоростей потоков на результат решения задачи

Сложность проблемы синхронизации кроется в нерегулярности возникающих ситуаций. Так, в предыдущем примере стр. 73 можно представить и другое развитие событий: могла быть потеряна информация не о заказе, а об оплате или, напротив, все исправления были успешно внесены. Все определяется взаимными скоростями потоков и моментами их прерывания. Поэтому отладка взаимодействующих потоков является сложной задачей, поскольку

ситуации, в которые «попадают» потоки могут меняться от запуска к запуску. Ситуации, подобные той, когда два или более потоков обрабатывают разделяемые данные и конечный результат зависит от соотношения скоростей потоков, называются *гонками*.

3.2. Критическая секция

Важным понятием синхронизации потоков является понятие «критической секции» программы. Критическая секция — это часть программы, результат выполнения которой может непредсказуемо меняться, если переменные, относящиеся к этой части программы, изменяются другими потоками в то время, когда выполнение этой части еще не завершено. Критическая секция всегда определяется по отношению к определенным критическим данным, при несогласованном изменении которых могут возникнуть нежелательные эффекты. В предыдущем примере такими критическими данными являлись записи файла базы данных. Во всех потоках, работающих с критическими данными, должна быть определена критическая секция. В разных потоках критическая секция состоит в общем случае из разных последовательностей команд.

Для исключения эффекта гонок по отношению к критическим данным, необходимо обеспечить, чтобы в каждый момент времени в критической секции, связанной с этими данными, находился только один поток. При этом неважно, находится этот поток в активном или в приостановленном состоянии. Этот прием называют взаимным исключением.

Задача взаимного исключения — фундаментальная по своему назначению задача. Любая ОС, управляющая параллельными процессами, должна обеспечить тот или иной вариант реализации ее решения. Суть задачи состоит в следующем: необходимо согласовать работу $n > 1$ параллельных потоков при использовании некоторого критического ресурса таким образом, чтобы удовлетворить следующим требованиям:

- одновременно внутри критической секции должно находиться не более одного потока;
- критические секции не должны иметь приоритеты в отношении друг друга;
- остановка какого-либо процесса вне его критической секции не должна влиять на дальнейшую работу процессов по использованию критического ресурса;

- решение о вхождении потоков в их критические секции при одинаковом времени поступления запросов на такое вхождение и равноприоритетности потоков не откладывается на неопределенный срок, а является конечным во времени;

- относительные скорости развития потоков неизвестны и произвольны;

- любой поток может переходить в любое состояние, отличное от активного, вне пределов своей критической секции;

- освобождение критического ресурса и выход из критической секции должны быть произведены потоком, использующим критический ресурс, за конечное время.

Операционная система использует разные способы реализации взаимного исключения. Некоторые способы пригодны для взаимного исключения при вхождении в критическую секцию только потоков одного процесса, в то время как другие могут обеспечить взаимное исключение и для потоков разных процессов. Самый простой и в то же время самый неэффективный способ обеспечения взаимного исключения состоит в том, что операционная система позволяет потоку запрещать любые прерывания на время его нахождения в критической секции. Однако этот способ практически не применяется, так как опасно доверять управление системой пользовательскому потоку – он может надолго занять процессор, а при крахе потока в критической секции крах потерпит вся система, потому что прерывания никогда не будут разрешены.

Для синхронизации потоков одного процесса прикладной программист может использовать глобальные блокирующие переменные. С этими переменными, к которым все потоки процесса имеют прямой доступ, программист работает, не обращаясь к системным вызовам ОС.

Каждому набору критических данных ставится в соответствие двоичная переменная. Поток присваивает ей значение 0, когда он входит в критическую секцию, и значение 1, когда ее покидает. На рис. 37 показан фрагмент алгоритма потока, использующего для реализации взаимного исключения доступа к критическим данным D блокирующую переменную $F(D)$. Перед входом в критическую секцию поток проверяет, не работает ли уже какой-нибудь поток с данными D . Если переменная $F(D)$ установлена в 0, то данные заняты, и проверка циклически повторяется. Если же данные свободны ($F(D) = 1$), то значение переменной $F(D)$ устанавливается

в 0 и поток входит в критическую секцию. После того как поток выполнит все действия с данными 0, значение переменной $F(D)$ снова устанавливается равным 1.

Блокирующие переменные могут использоваться не только при доступе к разделяемым данным, но и при доступе к разделяемым ресурсам любого вида.

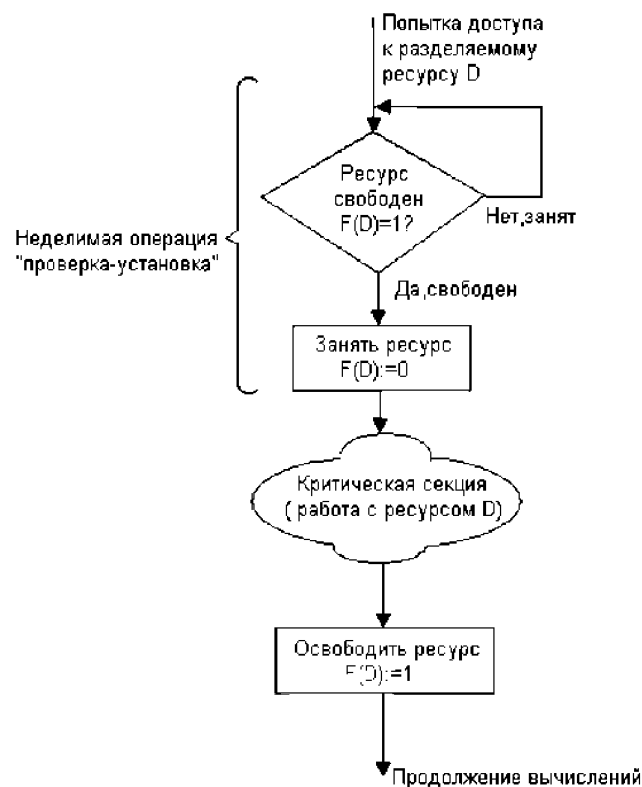


Рис. 37. Реализация критических секций с использованием блокирующих переменных

Если все потоки написаны с учетом вышеописанных соглашений, то взаимное исключение гарантируется. При этом потоки могут быть прерваны операционной системой в любой момент и в любом месте, в том числе в критической секции.

Однако одно ограничение на прерывания все же имеется. Нельзя прерывать поток между выполнением операций проверки и установки блокирующей переменной. Поясним это. Пусть в результате проверки переменной поток определил, что ресурс свободен, но сразу после этого, не успев установить переменную в 0, был прерван. За время его приостановки другой поток занял ресурс, вошел в свою критическую секцию, но также был прерван, не завершив работы с разделяемым ресурсом. Когда управление было возвращено

первому потоку, он, считая ресурс свободным, установил признак занятости и начал выполнять свою критическую секцию. Таким образом, был нарушен принцип взаимного исключения, что потенциально может привести к нежелательным последствиям. Во избежание таких ситуаций в системе команд многих компьютеров предусмотрена единая, неделимая команда анализа и присвоения значения логической переменной (например, команды BTC, BTR и BT5 процессора Pentium). При отсутствии такой команды в процессоре соответствующие действия должны реализовываться специальными системными примитивами, которые бы запрещали прерывания на протяжении всей операции проверки и установки.

Реализация взаимного исключения описанным выше способом имеет существенный недостаток: в течение времени, когда один поток находится в критической секции, другой поток, которому требуется тот же ресурс, получив доступ к процессору, будет непрерывно опрашивать блокирующую переменную, бесполезно тратя выделяемое ему процессорное время, которое могло бы быть использовано для выполнения какого-нибудь другого потока. Для устранения этого недостатка во многих ОС предусматриваются специальные системные вызовы для работы с критическими секциями. Перед тем как начать изменение критических данных, поток выполняет системный вызов EnterCriticalSection(). В рамках этого вызова сначала выполняется, как и в предыдущем случае, проверка блокирующей переменной, отражающей состояние критического ресурса. Если системный вызов определил, что ресурс занят ($F(D) = 0$), он в отличие от предыдущего случая не выполняет циклический опрос, а переводит поток в состояние ожидания (D) и делает отметку о том, что данный поток должен быть активизирован, когда соответствующий ресурс освободится. Поток, который в это время использует данный ресурс, после выхода из критической секции должен выполнить системную функцию LeaveCriticalSection(), в результате чего блокирующая переменная принимает значение, соответствующее свободному состоянию ресурса ($F(D) = 1$), а операционная система просматривает очередь ожидающих этот ресурс потоков и переводит первый поток из очереди в состояние готовности рис. 38.

Таким образом, исключается непроизводительная потеря процессорного времени на циклическую проверку освобождения занятого ресурса. Однако в тех случаях, когда объем работы

в критической секции небольшой и существует высокая вероятность в очень скором доступе к разделяемому ресурсу, более предпочтительным может оказаться использование блокирующих переменных. Действительно, в такой ситуации накладные расходы ОС по реализации функции входа в критическую секцию и выхода из нее могут превысить полученную экономию.

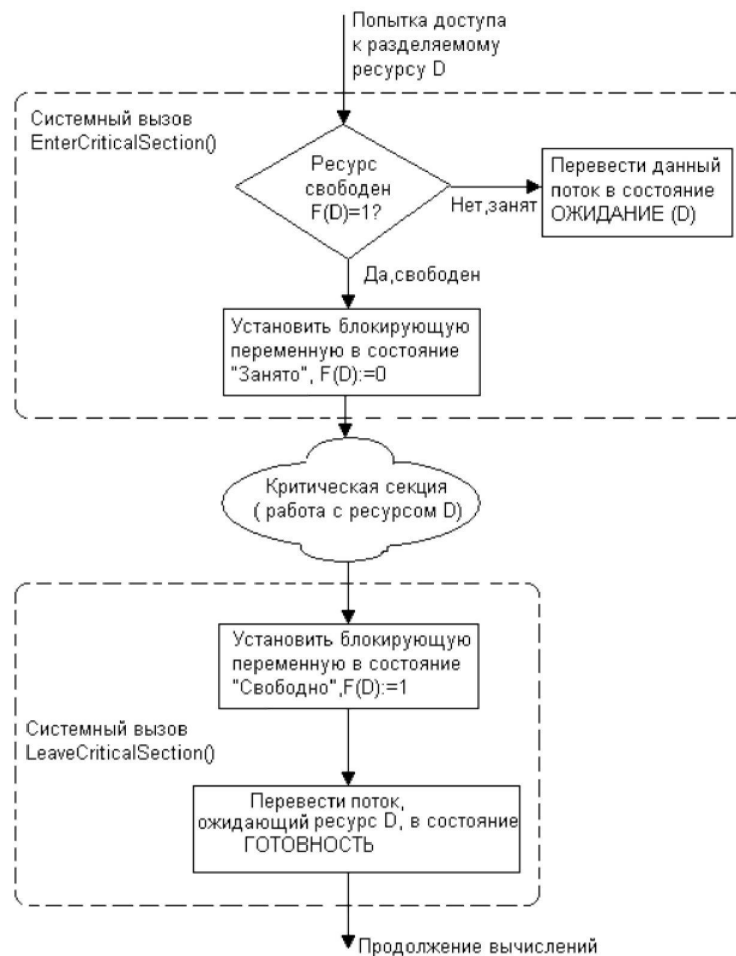


Рис. 38. Реализация взаимного исключения с использованием системных функций входа в критическую секцию и выхода из нее

3.3. Семафоры

Обобщением блокирующих переменных являются так называемые семафоры Дийкстры. Вместо двоичных переменных Дийкстра (Dijkstra) предложил использовать переменные, которые могут принимать целые неотрицательные значения. Такие

переменные, используемые для синхронизации вычислительных процессов, получили название семафоров.

Для работы с семафорами вводятся два примитива, традиционно обозначаемых P и V . Пусть переменная S представляет собой семафор. Тогда действия $V(S)$ и $P(S)$ определяются следующим образом.

– $V(S)$: переменная S увеличивается на 1 единым действием. Выборка, наращивание и запоминание не могут быть прерваны. К переменной S нет доступа другим потокам во время выполнения этой операции.

– $P(S)$: уменьшение S на 1, если это возможно. Если $S=0$ и невозможно уменьшить S , оставаясь в области целых неотрицательных значений, то в этом случае поток, вызывающий операцию P , ждет, пока это уменьшение станет возможным. Успешная проверка и уменьшение также являются неделимой операцией.

Никакие прерывания во время выполнения примитивов V и P недопустимы.

В частном случае, когда семафор S может принимать только значения 0 и 1, он превращается в блокирующую переменную, которую по этой причине часто называют двоичным семафором. Операция P включает в себе потенциальную возможность перехода потока, который ее выполняет, в состояние ожидания, в то время как операция V может при некоторых обстоятельствах активизировать другой поток, приостановленный операцией P .

Рассмотрим использование семафоров на классическом *примере* взаимодействия двух выполняющихся в режиме мультипрограммирования потоков, один из которых пишет данные в буферный пул, а другой считывает их из буферного пула (задача «писатель – читатель»). Пусть буферный пул состоит из N буферов, каждый из которых может содержать одну запись. В общем случае поток-писатель и поток-читатель могут иметь различные скорости и обращаться к буферному пулу с переменной интенсивностью. В один период скорость записи может превышать скорость чтения, в другой – наоборот. Для правильной совместной работы поток-писатель должен приостанавливаться, когда все буферы оказываются занятыми, и активизироваться при освобождении хотя бы одного буфера. Напротив, поток-читатель должен приостанавливаться, когда все буферы пусты, и активизироваться при появлении хотя бы одной записи.

Введем два семафора: e – число пустых буферов, и f – число заполненных буферов, причем в исходном состоянии $e=N$, а $f=0$. Тогда работа потоков с общим буферным пулом может быть описана следующим образом: поток-писатель прежде всего выполняет операцию $P(e)$, с помощью которой он проверяет, имеются ли в буферном пуле незаполненные буферы. В соответствии с семантикой операции P , если семафор e равен 0 (свободных буферов в данный момент нет), то поток-писатель переходит в состояние ожидания. Если же значением e является положительное число, то он уменьшает число свободных буферов, записывает данные в очередной свободный буфер и после этого наращивает число занятых буферов операцией $V(f)$. Поток-читатель действует аналогичным образом, с той разницей, что он начинает работу с проверки наличия заполненных буферов, а после чтения данных наращивает количество свободных буферов рис. 39.

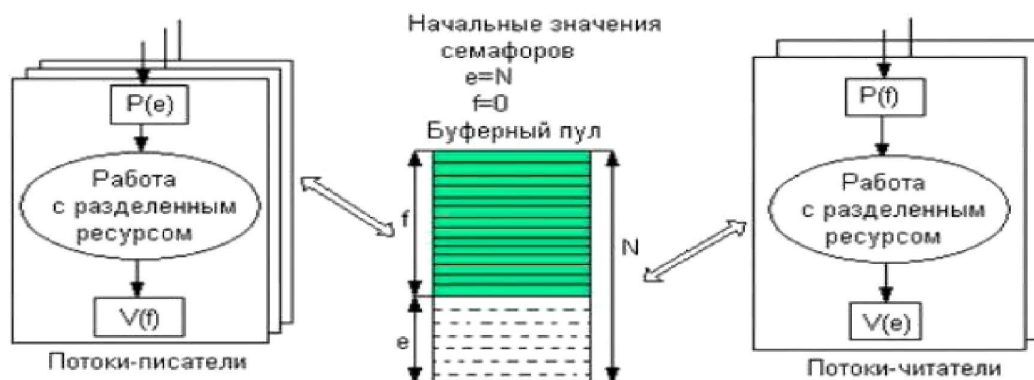


Рис. 39. Использование семафоров для синхронизации потоков

В данном случае предпочтительнее использовать семафоры вместо блокирующих переменных. Действительно, критическим ресурсом здесь является буферный пул, который может быть представлен как набор идентичных ресурсов — отдельных буферов, а значит, с буферным пулом могут работать сразу несколько потоков, и именно столько, сколько буферов в нем содержится. Использование двоичной переменной не позволяет организовать доступ к критическому ресурсу более чем одному потоку. Семафор же решает задачу синхронизации более гибко, допуская к разделяемому пулу ресурсов заданное количество потоков. Так, в нашем примере

с буферным пулом могут работать максимум N потоков, часть из которых может быть «писателями», а часть – «читателями».

Таким образом, семафоры позволяют эффективно решать задачу синхронизации доступа к ресурсным пулам, таким, например, как набор идентичных в функциональном назначении внешних устройств (модемов, принтеров, портов), или набор областей памяти одинаковой величины, или информационных структур. Во всех этих и подобных им случаях с помощью семафоров можно организовать доступ к разделяемым ресурсам сразу нескольких потоков.

Семафор может использоваться и в качестве блокирующей переменной. В рассмотренном выше примере (с. 79), для того чтобы исключить коллизии при работе с разделяемой областью памяти, будем считать, что запись в буфер и считывание из буфера являются критическими секциями. Взаимное исключение будем обеспечивать с помощью двоичного семафора b . Оба потока после проверки доступности буферов должны выполнить проверку доступности критической секции. На рис. 40 проиллюстрируем работу двоичного семафора.

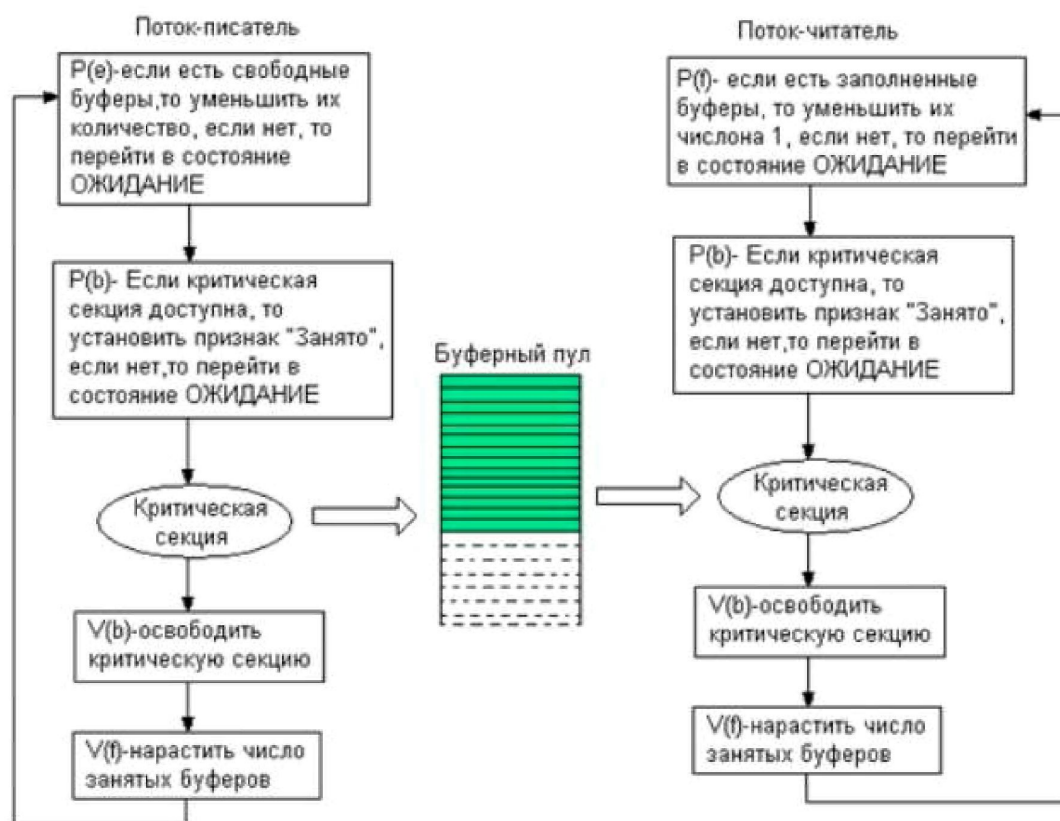


Рис. 40. Использование двоичного семафора

Тупики. Приведенный выше пример (с. 79) позволяет также проиллюстрировать еще одну проблему синхронизации – взаимные блокировки, называемые также дедлоками (deadlocks), клинчами (clinch), или тупиками. Покажем, что если переставить местами операции $P(e)$ и $P(b)$ в потоке-писателе, то при некотором стечении обстоятельств эти два потока могут взаимно блокировать друг друга. Итак, пусть поток-писатель начинает свою работу с проверки доступности критической секции — операции $P(b)$, и пусть он первым войдет в критическую секцию. Выполняя операцию $P(e)$, он может обнаружить отсутствие свободных буферов и перейти в состояние ожидания. Как уже было показано, из этого состояния его может вывести только поток-читатель, который возьмет очередную запись из буфера. Однако поток-читатель не сможет этого сделать, так как для этого ему потребуется войти в критическую секцию, вход в которую заблокирован потоком-писателем. Таким образом, ни один из этих потоков не может завершить начатую работу и возникнет тупиковая ситуация, которая не может разрешиться без внешнего воздействия.

Пример тупика. Пусть двум потокам, принадлежащим разным процессам и выполняющимся в режиме мультипрограммирования, для выполнения их работы нужно два ресурса, например принтер и последовательный порт. Такая ситуация может возникнуть, например, во время работы приложения, задачей которого является распечатка информации, поступающей по модемной связи.

На рис. 41 показаны фрагменты соответствующих программ. Поток А запрашивает сначала принтер, а затем порт, а поток В запрашивает устройства в обратном порядке. Предположим, что после того, как ОС назначила принтер потоку А и установила связанную с этим ресурсом блокирующую переменную, поток А был прерван. Управление получил поток В, который сначала выполнил запрос на получение СОМ-порта, затем при выполнении следующей команды был заблокирован, так как принтер оказался уже занятым потоком А. Управление снова получил поток А, который в соответствии со своей программой сделал попытку занять порт и был заблокирован, поскольку порт уже выделен потоку В. В таком положении потоки А и В могут находиться сколь угодно долго.

В зависимости от соотношения скоростей потоков они могут либо взаимно блокировать друг друга, либо образовывать очереди к разделяемым ресурсам, либо совершенно независимо использовать разделяемые ресурсы.

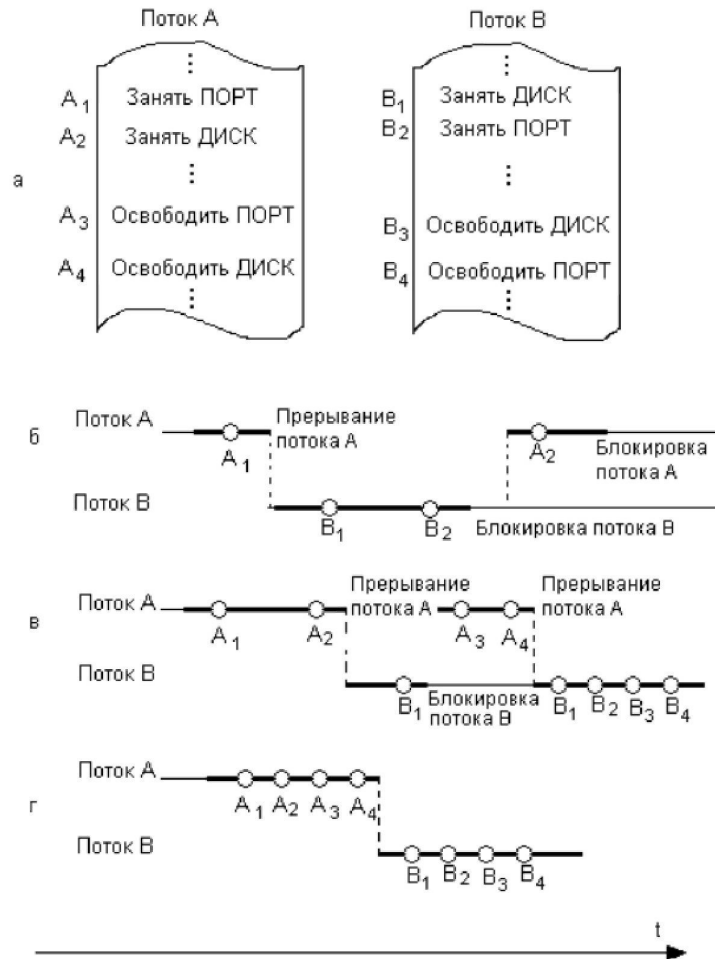


Рис. 41. Возникновение взаимных блокировок при выполнении программы

В рассмотренных примерах тупик был образован двумя потоками, но взаимно блокировать друг друга может и большее число потоков. Возможно такое распределение ресурсов R_i между несколькими потоками T_j , которое приводит к возникновению взаимных блокировок. Стрелки обозначают потребность потока в ресурсах. Сплошная стрелка означает, что соответствующий ресурс был выделен потоку, а пунктирная стрелка соединяет поток с тем ресурсом, который необходим, но не может быть пока выделен, поскольку занят другим потоком. Например, потоку T_1 для выполнения работы необходимы ресурсы R_1 и R_2 , из которых выделен только один —

R_1 , а ресурс R_2 удерживается потоком T_2 . Ни один из четырех показанных на рис. 42 потоков не может продолжить свою работу, так как не имеет всех необходимых для этого ресурсов.

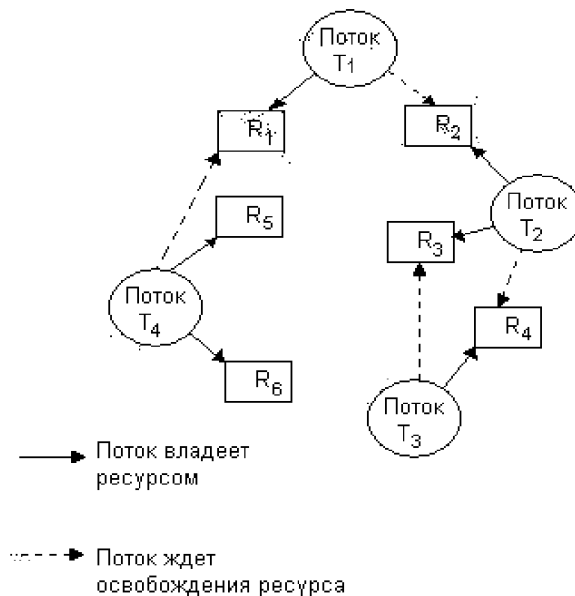


Рис. 42. Взаимная блокировка нескольких потоков

Невозможность потоков завершить начатую работу из-за возникновения взаимных блокировок снижает производительность вычислительной системы. Поэтому проблеме предотвращения тупиков уделяется большое внимание. На тот случай, когда взаимная блокировка все же возникает, система должна предоставить администратору средства, с помощью которых он смог бы распознать тупик, отличить его от обычной блокировки из-за временной недоступности ресурсов. И, наконец, если тупик диагностирован, то нужны средства для снятия взаимных блокировок и восстановления нормального вычислительного процесса.

Тупики могут быть предотвращены на стадии написания программ, т.е. программы должны быть написаны таким образом, чтобы тупик не мог возникнуть при любом соотношении взаимных скоростей потоков. Так, если бы в рассмотренном ранее примере поток А и поток В запрашивали ресурсы в одинаковой последовательности, то тупик был бы в принципе невозможен. Другой, более гибкий подход к предотвращению тупиков заключается в том, что ОС каждый раз при запуске задач анализирует их потребности в ресурсах и определяет, может ли в данной мультипрограммной смеси возникнуть тупик. Если да, то запуск новой задачи временно откла-

дывается. Операционная система может также использовать определенные правила при назначении ресурсов потокам, например, ресурсы могут выделяться операционной системой в определенной последовательности, общей для всех потоков.

В тех же случаях, когда тупиковую ситуацию не удалось предотвратить, важно быстро и точно ее распознать, поскольку заблокированные потоки не выполняют никакой полезной работы. Если тупиковая ситуация образована множеством потоков, занимающих массу ресурсов, распознавание тупика является непростой задачей. Существуют формальные, программно реализованные методы распознавания тупиков, основанные на ведении таблиц распределения ресурсов и таблиц запросов к занятым ресурсам. Анализ этих таблиц позволяет обнаружить взаимные блокировки.

Если же тупиковая ситуация возникла, то не обязательно снимать с выполнения все заблокированные потоки. Можно снять только часть из них, освободив ресурсы, ожидаемые остальными потоками, можно вернуть некоторые потоки в область подкачки, можно совершить «откат» некоторых потоков до так называемой контрольной точки, в которой запоминается вся информация, необходимая для восстановления выполнения программы с данного места. Контрольные точки расставляются в программе в тех местах, после которых возможно возникновение тупика.

3.4. Синхронизирующие объекты ОС

Рассмотренные в п. 3.3 механизмы синхронизации, основанные на использовании глобальных переменных процесса, обладают существенным недостатком – они не подходят для синхронизации потоков разных процессов. В таких случаях операционная система должна предоставлять потокам системные объекты синхронизации, которые были бы видны для всех потоков, даже если они принадлежат разным процессам и работают в разных адресных пространствах.

Примерами таких синхронизирующих объектов ОС являются системные семафоры, мьютексы, события, таймеры и другие – их набор зависит от конкретной ОС, которая создает эти объекты по запросам процессов. Для того чтобы процессы могли разделять синхронизирующие объекты, в разных ОС используются разные методы. Некоторые ОС возвращают указатель на объект. Этот указатель может быть доступен всем родственным процессам, наследую-

щим характеристики общего родительского процесса. В других ОС процессы в запросах на создание объектов синхронизации указывают имена, которые должны быть им присвоены. Далее эти имена используются разными процессами для манипуляций объектами синхронизации. В таком, случае работа с синхронизирующими объектами подобна работе с файлами. Их можно создавать, открывать, закрывать, уничтожать.

Кроме того, для синхронизации могут быть использованы такие «обычные» объекты ОС, как файлы, процессы и потоки. Все эти объекты могут находиться в двух состояниях: сигнальном и несигнальном (свободном). Для каждого объекта смысл, вкладываемый в понятие «сигнальное состояние», зависит от типа объекта. Так, например, поток переходит в сигнальное состояние тогда, когда он завершается. Процесс переходит в сигнальное состояние тогда, когда завершаются все его потоки. Файл переходит в сигнальное состояние в том случае, когда завершается операция ввода-вывода для этого файла. Для остальных объектов сигнальное состояние устанавливается в результате выполнения специальных системных вызовов. Приостановка и активизация потоков осуществляются в зависимости от состояния синхронизирующих объектов ОС.

Потоки с помощью специального системного вызова сообщают операционной системе о том, что они хотят синхронизировать свое выполнение с состоянием некоторого объекта. Будем далее называть этот системный вызов `Wait(X)`, где `X` — указатель на объект синхронизации. Системный вызов, с помощью которого поток может перевести объект синхронизации в сигнальное состояние, назовем `Set(X)` [2].

Поток, выполнивший системный вызов `Wait(X)`, переводится операционной системой в состояние ожидания до тех пор, пока объект `X` не перейдет в сигнальное состояние. Примерами системных вызовов типа `Wait()` и `Set()` являются вызовы `WaitForSingleObject()` и `SetEvent()` в Windows NT, `DosSemWait()` и `OosSemSet()` в OS/2, `sleep()` и `wakeup()` в UNIX.

Поток может ожидать установки сигнального состояния не одного объекта, а нескольких. При этом поток может попросить ОС активизировать его при установке либо одного из указанных объектов, либо всех объектов. Поток может в качестве аргумента системного вызова `Wait()` указать также максимальное время, которое он будет ожидать перехода объекта в сигнальное состояние, после

чего ОС должна его активизировать в любом случае. Может случиться, что установки некоторого объекта в сигнальное состояние ожидают сразу несколько потоков. В зависимости от объекта синхронизации в состояние готовности могут переводиться либо все ожидающие это событие потоки, либо один из них.

Синхронизация тесно связана с планированием потоков. Во-первых, любое обращение потока с системным вызовом Wait(X) влечет за собой действия в подсистеме планирования – этот поток снимается с выполнения и помещается в очередь ожидающих потоков, а из очереди готовых потоков выбирается и активизируется новый поток. Во-вторых, при переходе объекта в сигнальное состояние (в результате выполнения некоторого потока – либо системного, либо прикладного) ожидающий этот объект поток (или потоки) переводится в очередь готовых к выполнению потоков. В обоих случаях осуществляется перепланирование потоков, при этом если в ОС предусмотрены изменяемые приоритеты и/или кванты времени, то они пересчитываются по правилам, принятым в этой операционной системе.

Примеры, в которых в качестве синхронизирующих объектов используются файлы, потоки и процессы.

1. Пусть программа приложения построена так, что для выполнения запросов, поступающих из сети, основной поток создает вспомогательные серверные потоки.

При поступлении от пользователя команды завершения приложения основной поток должен дожидаться завершения всех серверных потоков и только после этого завершиться сам. Следовательно, процедура завершения должна включать вызов Wait(X1, X2, ...), где X1, X2 – указатели на серверные потоки. В результате выполнения данного системного вызова основной поток будет переведен в состояние ожидания и останется в нем до тех пор, пока все серверные потоки не перейдут в сигнальное состояние, т.е. завершатся. После этого ОС переведет основной поток в состояние готовности. При получении доступа к процессору основной поток завершится.

2. Пусть выполнение некоторого приложения требует последовательных работ – этапов. Для каждого этапа имеется свой отдельный процесс. Сигналом для начала работы каждого следующего процесса является завершение предыдущего. Для реализации такой логики работы необходимо в каждом процессе, кроме первого, предус-

мотреть выполнение системного вызова `Wait(X)`, в котором синхронизирующим объектом является предшествующий поток.

Объект-файл, переход которого в сигнальное состояние соответствует завершению операции ввода-вывода с этим файлом, используется в тех случаях, когда поток, инициировавший эту операцию, решает дождаться ее завершения, прежде чем продолжить свои вычисления.

Однако круг событий, с которыми потоку может потребоваться синхронизировать свое выполнение, отнюдь не исчерпывается завершением потока, процесса или операции ввода-вывода. Поэтому в ОС, как правило, имеются и другие, более универсальные объекты синхронизации, такие как событие (`event`), мьютекс (`nmtx`), системный семафор и другие.

Мьютекс, как и семафор, обычно используется для управления доступом к данным.

В отличие от объектов-поток, объектов-процессов и объектов-файлов, которые при переходе в сигнальное состояние переводят в состояние готовности все потоки, ожидающие этого события, объект-мьютекс «освобождает» из очереди ожидающих только один поток.

Работа мьютекса хорошо поясняется в терминах «владения». Пусть поток, который, пытаясь получить доступ к критическим данным, выполнил системный вызов `Wait(X)`, где `X` – указатель на мьютекс. Предположим, что мьютекс находится в сигнальном состоянии, в этом случае поток тут же становится его владельцем, устанавливая его в несигнальное состояние, и входит в критическую секцию. После того как поток выполнил работу с критическими данными, он «отдает» мьютекс, устанавливая его в сигнальное состояние. В этот момент мьютекс свободен и не принадлежит ни одному потоку. Если какой-либо поток ожидает его освобождения, то он становится следующим владельцем этого мьютекса, одновременно мьютекс переходит в несигнальное состояние.

Объект-событие (в данном случае слово «событие» используется в узком смысле, как обозначение конкретного вида объектов синхронизации) обычно используется не для доступа к данным, а для того, чтобы оповестить другие потоки о том, что некоторые действия завершены. Пусть, например, в некотором приложении работа организована таким образом, что один поток читает данные из файла в буфер памяти, а другие потоки обрабатывают эти данные, затем

первый поток считывает новую порцию данных, а другие потоки снова ее обрабатывают и так далее. В начале работы первый поток устанавливает объект-событие в несигнальное состояние. Все остальные потоки выполнили вызов `Wait(X)`, где `X` – указатель события, и находятся в приостановленном состоянии, ожидая наступления этого события. Как только буфер заполняется, первый поток сообщает об этом операционной системе, выполняя вызов `Set(X)`. Операционная система просматривает очередь ожидающих потоков и активизирует все потоки, которые ждут этого события.

3.5. Сигналы

Сигнал дает возможность задаче реагировать на событие, источником которого может быть операционная система или другая задача. Сигналы вызывают прерывание задачи и выполнение заранее предусмотренных действий. Сигналы могут вырабатываться синхронно (как результат работы самого процесса), а могут быть направлены процессу другим процессом (вырабатываться асинхронно). Синхронные сигналы чаще всего приходят от системы прерываний процессора и свидетельствуют о действиях потока процесса, блокируемых аппаратурой, например деление на ноль, ошибка адресации, нарушение защиты памяти и т.д.

Примером асинхронного сигнала является сигнал с терминала. Во многих ОС предусматривается оперативное снятие процесса с выполнения. Для этого пользователь может нажать некоторую комбинацию клавиш (`Ctrl+C`, `Ctrl+Break`) [2], в результате чего ОС вырабатывает сигнал и направляет его активному процессу. Сигнал может поступить в любой момент выполнения процесса (он является асинхронным), требуя от процесса немедленного завершения работы. В данном случае реакцией на сигнал является безусловное завершение процесса.

В системе может быть определен набор сигналов. Программный код процесса, которому поступил сигнал, может либо проигнорировать его, либо прореагировать на него стандартным действием (например, завершиться), либо выполнить специфические действия, определенные прикладным программистом. В последнем случае в программном коде необходимо предусмотреть специальные системные вызовы, с помощью которых операционная система инфор-

мируется, какую процедуру надо выполнить в ответ на поступление того или иного сигнала.

Сигналы обеспечивают логическую связь между процессами, а также между процессами и пользователями (терминалами). Поскольку посылка сигнала предусматривает знание идентификатора процесса, то взаимодействие посредством сигналов возможно только между родственными процессами, которые могут получить данные об идентификаторах друг друга.

В распределенных системах, состоящих из нескольких процессоров, каждый из которых имеет собственную оперативную память, блокирующие переменные, семафоры, сигналы и другие аналогичные средства, основанные на разделяемой памяти, оказываются непригодными. В таких системах синхронизация может быть реализована только посредством обмена сообщениями.

3.6. Обмен сообщениями

Цели данной модели – избавиться от проблем разделения памяти и предложить модель взаимодействия процессов для распределенных систем. Предлагаются две операции:

send (destination, &message, msize) передача сообщения.

Receive ([source], &message, msize) прием сообщения.

В качестве адресата выступает процесс. Отправитель может не специфицироваться (например, быть любым – широковещательное сообщение). Передача может осуществляться как с буферизацией (почтовые ящики) так и без нее (рандеву – Ада, Оккам). Пайпы ОС UNIX – почтовые ящики, заменяют файлы и не хранят границы сообщений (все сообщения объединяются в одно большое, которое можно читать произвольными порциями).

Механизмы семафоров и обмена сообщениями взаимозаменяемы семантически и на мультипроцессорах могут быть реализованы один через другой.

3.7. Реализация взаимного исключения

Алгоритм Петерсона. Первое решение проблемы взаимного исключения, удовлетворяющее всем требованиям, было предложено датским математиком Т. Деккером (T. Dekker). В 1981 г. Г. Петерсон (G. Peterson) предложил более изящное решение.

Пусть два потока имеют доступ к массиву флагов готовности и к переменной очередности. Предлагается использовать следующий код:

```
int ready[2] = {0, 0}; /* разделяемая переменная */
int turn;               /* разделяемая переменная */
while(some_condition) {
    ready[i] = 1;
    turn = 1 - i;
    while( ready[1-i] && turn == 1-i )
        ;
    critical section
    ready[i] = 0;
    remainder section
}
```

При исполнении пролога критической секции поток P_1 заявляет о своей готовности выполнить критический участок и одновременно предлагает другому потоку приступить к его выполнению. Если оба потока подошли к прологу практически одновременно, то они оба объявят о своей готовности и предложат выполняться друг другу. При этом одно из предложений всегда последует после другого. Тем самым работу в критическом участке продолжит поток, которому было сделано последнее предложение.

Поскольку в процессе ожидания разрешения на вход поток P_0 не изменяет значения переменных, то он сможет начать исполнение своего критического участка после не более чем одного прохода по критической секции потока P_1 .

Алгоритм булочной (Bakery algorithm). Алгоритм Петерсона дает нам решение задачи корректной организации взаимодействия двух потоков. Рассмотрим теперь соответствующий алгоритм для n взаимодействующих потоков, который получил название алгоритм булочной. Основная его идея выглядит так. Каждый вновь прибывающий клиент (он же поток) получает талончик на обслуживание с номером. Клиент с наименьшим номером на талончике обслуживается следующим. К сожалению, из-за неатомарности операции вычисления следующего номера алгоритм булочной

не гарантирует, что у всех потоков будут талончики с разными номерами. В случае равенства номеров на талончиках у двух или более клиентов первым обслуживается клиент с меньшим значением имени (имена можно сравнивать в лексикографическом порядке). Разделяемые структуры данных для алгоритма – это два массива

```
enum {false, true} choosing[n];
int number[n].
```

Изначально элементы этих массивов инициализируются значениями false и 0 соответственно. Введем следующие обозначения:

$(a, b) < (c, d)$, если $a < c$ или если $a = c$ и $b < d$;

$\max(a_0, a_1, \dots, a_n)$ – это число k такое, что $k \geq a_i$ для всех $i = 0, \dots, n$.

Структура кода потока P_i для алгоритма булочной:

```
while(some_condition) {
    choosing[i] = true;
    number[i] = max(number[0], ..., number[n-1]) + 1; choosing[i] = false;
    for( j = 0; j < n; j++ ){ while(choosing[j]);
    while(number[j] != 0 && (number[j], j) < (number[i], i));
    }
    critical section
    number[i] = 0;
    remainder section
}
```

3.8. Аппаратная поддержка взаимного исключения

Наличие аппаратной поддержки взаимного исключения позволяет упростить алгоритмы и повысить их эффективность точно также, как это происходит и в других областях программирования [4]. Многие вычислительные системы помимо этого имеют специальные команды процессора, которые позволяют проверить и изменить значение машинного слова или поменять местами значения двух машинных слов в памяти, выполняя эти действия как атомарные операции. Рассмотрим, как концепции таких команд могут быть использованы для реализации взаимного исключения [5].

Команда Test-and-Set (Проверить и присвоить 1)

О выполнении команды Test-and-Set, осуществляющей проверку значения логической переменной с одновременной установкой ее значения в 1, можно думать, как об атомарном выполнении следующей функции:

```
int Test_and_Set (int*target){  
    int tmp = *target; *target = 1;  
    return tmp;  
}
```

С использованием этой атомарной команды можно записать следующий алгоритм, обеспечивающий взаимное исключение:

```
int lock = 0; /* Общая переменная*/  
while(some_condition) {  
    while(Test_and_Set(&lock))  
        ;  
    critical section  
    lock = 0;  
    remainder section  
}
```

К сожалению, даже в таком виде полученный алгоритм не удовлетворяет условию ограниченного ожидания.

Команда Swap (Обменять значения)

Выполнение команды Swap, обменивающей два значения, находящихся в памяти, можно проиллюстрировать следующей функцией:

```
void Swap (int *a, int *b){  
    int tmp = *a;  
    *a = *b;  
    *b = tmp;  
}
```

Применяя атомарную команду Swap, можно реализовать предыдущий алгоритм, введя дополнительную логическую переменную key локальную для каждого процесса (потока):

```
shared int lock = 0; /* Общая переменная*/ int key;  
while (some condition) {  
    key = 1;  
    do{  
        Swap(&lock,&key); }  
    while (key);
```

```
critical section
lock = 0;
remainder section
}
```

Приведенный вариант также не удовлетворяет условию ограниченного ожидания, однако и в этом, и в предыдущем случае условие выполняется после небольшой модификации алгоритма.

3.9. Классические задачи взаимодействия потоков

Задача «Производитель-потребитель»

Имеется большое число вариантов постановки и решения такой задачи в рамках конкретных ОС. Наиболее простой случай, когда взаимодействуют два потока с жестко распределенными между ними функциями. Один поток вырабатывает сообщения, предназначенные для восприятия и обработки другим потоком. Поток, вырабатывающий сообщения, называют производителем, а воспринимающий сообщения – потребителем. Потоки взаимодействуют через некоторую обобщенную область памяти, которая по смыслу является критическим ресурсом. В эту область поток-производитель должен помещать очередное сообщение (в простейшем случае предполагается, что область способна хранить только одно сообщение и сообщения фиксированной длины), а поток-потребитель должен считывать очередное сообщение. Необходимо согласовать работы двух потоков при одностороннем (в простейшем случае) обмене сообщениями по мере развития потоков таким образом, чтобы удовлетворить следующим требованиям:

- выполнять требования задачи взаимного исключения по отношению к критическому ресурсу – обобщенной памяти для хранения сообщения;

- учитывать состояние обобщенной области памяти, характеризующей возможность или невозможность отправки (принятия) очередного сообщения.

Попытка потока-производителя поместить очередное сообщение в область, из которой не было считано предыдущее сообщение потоком-потребителем, должна быть блокирована. Поток-производитель должен быть либо оповещен о невозможности помещения сообщения, либо переведен в состояние ожидания возможности

поместить очередное сообщение через некоторое время в область памяти, по мере ее освобождения. Аналогично должна быть блокирована попытка потока-потребителя считать сообщение из области в ситуации, когда поток-производитель не поместил туда очередного сообщения. Возможны также несколько вариантов реакции на данную ситуацию – сообщение о невозможности считывания либо перевод потока-потребителя в состояние ожидания поступления очередного сообщения. Если используется вариант с ожиданиями изменения состояния обобщенной области для хранения сообщения, необходимо обеспечить перевод ожидающих потоков в состояние готовности всякий раз, когда изменится состояние области. Либо поток-производитель поместит очередное сообщение в область. Оно теперь может быть считано ожидающим потоком-потребителем. Либо поток-потребитель считал очередное сообщение из области и обеспечил возможность ожидающему потоку-производителю послать очередное сообщение.

Множественность постановки задачи «производитель-потребитель» определяется тем, что число как потоков-потребителей, так и потоков-производителей может быть больше одного. Каждый из таких процессов может устанавливать не только одностороннюю, но и двустороннюю связь через одну и ту же обобщенную область или другие области. Области могут хранить не одно, а большее количество сообщений.

Задача «Читатели – писатели»

Эта задача также имеет много вариантов решения. Наиболее характерная область использования этой задачи – при построении файловых систем ОС. В отношении некоторой области памяти, являющейся по смыслу критическим ресурсом для параллельных потоков, работающих с ней, выделяется два типа потоков.

Первый тип – потоки-читатели. Они считывают одновременно информацию из области, если это допускается при работе с конкретным устройством памяти.

Второй тип – потоки-писатели, которые записывают информацию в область и могут делать это только исключая и друг друга, и потоки-читатели, т.е. запись должна удовлетворяться на основе решения задачи взаимного исключения. Имеются различные варианты взаимодействия между потоками-писателями и потоками-читателями. Наиболее широко распространены следующие варианты:

1. Устанавливается приоритетность в использовании критического ресурса потокам-читателям. Если хотя бы один поток-читатель пользуется ресурсом, то он закрыт для использования всем потокам-писателям и доступен для использования всем потокам-читателям.

2. Большой приоритет имеют потоки-писатели. При появлении запроса от потока-писателя необходимо закрыть ресурс для использования всем потокам-читателям, которые выдадут запрос позже него.

Пример решения задачи программирования взаимодействия множества потоков с использованием семафоров. Пусть выполняются n потоков, которые разделяют некоторый объект программы, например переменную. Одни потоки (писатели) только модифицируют разделяемый объект, другие (читатели) – только считывают значение. Необходимо организовать корректное выполнение этой системы взаимодействующих потоков.

Прежде всего, потоки-читатели могут иметь одновременный доступ к разделяемому объекту, так как чтение не меняет значение объекта и, следовательно, потоки-читатели не могут влиять друг на друга. Для потоков-писателей следует организовать взаимное $\square$ сключение при доступе к разделяемому объекту.

Далее, потоки-писатели должны иметь преимущество при доступе к разделяемому объекту, поскольку они готовы записать новые данные. Следовательно, потоки-читатели должны получать доступ к разделяемому объекту лишь в том случае, если нет потоков-писателей, желающих получить доступ к этому разделяемому объекту. Поэтому, если поток-писатель изъявил желание получить доступ к разделяемому объекту, все потоки-читатели должны быть задержаны. С другой стороны, потоки-читатели тоже будет неправильно оставлять надолго в процессе ожидания и поэтому потоки-читатели, изъявившие желание получить доступ к разделяемому объекту до того, как появился первый поток-писатель, должны иметь возможность завершить исполнение своего критического интервала.

Одно из решений задачи выглядит следующим образом.

Семафор `Mutex = 1;` // Контроль доступа к переменной
`ReadersCount`

Семафор `Access = 1;` // Контроль доступа к данным

`int ReadersCount = 0;` // Число читателей

`void Reader( void ){`

`while( true ){`

`P(Mutex);` // Получение монопольного доступа к RC

```

ReadersCount ++; // Увеличение числа читателей
if( ReadersCount == 1 ) // Если это первый читатель
P(Access); // Захватываем доступ для читателей
V(Mutex); // Отказ от монопольного доступа к RC
... Доступ к данным ...
P(Mutex); // Получение монопольного доступа к RC
ReadersCount --; // Уменьшение числа читателей
if( ReadersCount == 0 ) // Если это последний читатель
V(Access); // Открываем доступ для всех
V(Mutex); // Отказ от монопольного доступа к RC
... Работа вне критической области ...
}
}
void Writer( void ){
while( true ){
... Формирование данных ...
P(Access); // Захватываем доступ для себя
... Запись данных ...
V(Access); // Открываем доступ для всех
}
}

```

Данное решение справедливо для случая привилегированных потоков-читателей.

Задача «Обедающие философы»

Хотя и название, и формулировка задачи носят некоторый абстрактный характер, такая задача синхронизации также имеет место при построении систем распределения ресурсов в составе ОС. В рамках этой задачи формулируются требования на синхронизацию работы потоков, которые совместно используют пересекающиеся группы ресурсов.

Обратимся к формулировке задачи, изложенной в терминологии, предложенной впервые Э. Дейкстрой.

За круглым столом расставлены стулья, каждый из которых занимает определенный философ (в первой формулировке Э. Дейкстры число философов равнялось пяти). В центре стола — большое блюдо спагетти, а на столе лежат пять вилок — каждая между двумя соседними тарелками. Каждый философ находится только в двух состояниях — либо он размышляет, либо ест спагетти. Начать думать после еды философу ничто не мешает. Однако чтобы начать

есть, необходимо выполнить некоторые условия. Предполагается, что любой философ, прежде чем начать есть, должен положить из общего блюда спагетти себе в тарелку. Для этого он одновременно должен держать в левой и правой руках по вилке, набрать спагетти в тарелку с их помощью и, не выпуская вилок из рук, начать есть. Закончив еду, философ кладет вилки слева и справа от своей тарелки и опять начинает размышлять до тех пор, пока снова проголодается.

Нетрудно заметить, что вилки в данной «столовой ситуации» выступают в качестве пересекающихся ресурсов. Неприятная ситуация может возникнуть в случае, когда философы одновременно проголодаются и одновременно попытаются взять, например, свою левую вилку. В данном случае возникает тупиковая ситуация, так как никто из них не может по условию начать есть, не имея второй вилки. Однако вторая вилка может появиться для любого философа только от соседа справа, который в свою очередь ждет появления для себя вилки от своего соседа справа и одновременно не хочет положить свою левую вилку на стол и т.д.

Вторая неприятная ситуация (голодание) может возникнуть уже в отношении не всех, а только одного процесса. В обществе философов такая ситуация возникает в случае заговора двух соседей слева и справа против философа, в отношении которого строятся козни. Каждый раз, когда последний желает утолить голод, заговорщики, опережая его, поочередно забирают вилки то слева, то справа от него. Такие согласованные действия злоумышленников приводят жертву к вынужденному голоданию, так как он никогда не может воспользоваться обеими вилками.

Предлагаемое решение исключает взаимоблокировку и позволяет реализовать максимально возможный параллелизм. Состояние философа: ест, размышляет или голодает (пытаясь получить вилки) – отражены в массиве state.

```
#define LEFT (i+4)%5
#define RIGHT (i+1)%5
#define THINKING 0
#define HUNGRY 1
#define EATING 2
int state[5]; // Массив состояний философов
СемафорMutex; // Взаимное исключение для критических областей
СемафорS[5]; // Каждому философу по семафору
```

```

void Philosopher( int i ){ // i – номер философа
while( true ){
think(); // Философ размышляет
take_forks(i); // Получает две вилки или блокируется
eat(); // Философ обедает
put_forks(i); // Кладет на стол обе вилки
}
}
void take_forks( int i){
P(Mutex); // Вход в критическую область
state[i] = HUNGRY; // Наличие голодного философа
test(i); // Попытка получить две вилки
V(Mutex); // Выход из критической области
P(s[i]); // Блокируемся, если вилок не досталось
}
void put_forks( int i){
P(Mutex); // Вход в критическую область
state[i] = THINKING; // Наличие голодного философа
test(LEFT); // Попытка накормить соседа слева
test(RIGHT); // Попытка накормить соседа справа
V(Mutex); // Выход из критической области
}
void test( int i ){
if(state[i]==HUNGRY&&state[LEFT]!=EATING&&
state[RIGHT]!=EATING){
state[i] = EATING;
V(s[i]);
}
}

```

«Проблема спящего брадобрея»

Данная задача формулируется следующим образом.

В парикмахерской есть один брадобрей, его кресло и N стульев для посетителей. Если желающих воспользоваться его услугами нет, брадобрей сидит и спит. Если в парикмахерскую приходит клиент, он должен разбудить брадобрея. Если клиент приходит и видит, что брадобрей занят, он либо садится на стул (если есть место), либо уходит (если места нет).

В предлагаемом решении используется три семафора: Customers, для подсчета ожидающих посетителей (клиент в кресле бравобреля не учитывается, поскольку он уже не ждет), Barbers – количество бравобреев, простаивающих в ожидании клиента (0 или 1), и Mutex для реализации взаимного исключения.

```
#define CHAIRS 5
Семафор Customers = 0; // Количество ожидающих посетителей
Семафор Barbers = 0;    // Количество ожидающих бравобреев
Семафор Mutex = 1;
int waiting = 0;
void barber( void ){
while( true ){
P(Customers); // Ждать посетителей
P(Mutex);     // Запрос доступа к переменной waiting
waiting --;   // Уменьшение числа ожидающих клиентов
V(Barbers);   // Один бравобрей готов к работе
V(Mutex);     // Отказ от доступа к переменной waiting
... Постричь клиента ...
}
}
void customer( void ){
P(Mutex);     // Запрос доступа к переменной waiting
if( waiting < CHAIRS ){ // Есть ли свободные стулья?
Waiting ++; // Увеличение числа ожидающих клиентов
V(Customers); // При необходимости, разбудить бравобреля
V(Mutex);     // Отказ от доступа к переменной waiting
P(Barbers);   // Ждать освобождения бравобреля
... Постричься ...
}
else{
V(Mutex);     // Много посетителей, придется уйти
}
}
```

Помимо представленных существует много задач с экзотическими названиями и содержанием, но имеющих, тем не менее, совершенно определенную практическую значимость при управлении процессами и распределении ресурсов.

3.10. Передача данных между взаимодействующими потоками

При взаимодействии потокам необходимо передавать друг другу данные. По объему передаваемой информации и степени возможного воздействия на поведение другого потока все средства такого обмена можно условно разделить на три категории.

1. Сигнальные. Передается минимальное количество информации – один бит, «да» или «нет». Используются, как правило, для извещения потока о наступлении какого-либо события. Сигналы, очевидно, могут одновременно использоваться и для синхронизации выполнения потоков.

2. Канальные. Один поток может последовательно передавать другому данные. Передача может осуществляться единым потоком или в виде последовательности сообщений. Объем передаваемой информации в единицу времени ограничен пропускной способностью. Передача данных может быть синхронной и асинхронной. Примеры реализаций: именованные и неименованные каналы, очереди сообщений, сокет. Очевидно, канальные средства передачи данных также могут использоваться для синхронизации выполнения.

3. Разделяемая память. Два или более процессов могут совместно использовать некоторую область адресного пространства (в случае нескольких потоков одного процесса они разделяют память этого процесса изначально). Созданием и поддержкой разделяемой памяти занимается операционная система. Возможность обмена информацией подобным способом максимальна. Разделяемая память представляет собой наиболее быстрый способ взаимодействия процессов в одной вычислительной системе.

При взаимодействии потоков основной проблемой, с которой приходится сталкиваться, является синхронизация их исполнения. Передача данных между потоками – технический вопрос и требует в большинстве случаев изучения системных вызовов, реализованных в конкретной операционной системе. Исключение составляют случаи, когда механизм обмена данными обладает синхронизирующими свойствами. В этом случае пригодны все рассуждения, приведенные при описании использования синхронизирующих объектов.

4. АРХИТЕКТУРА ФАЙЛОВОЙ СИСТЕМЫ

4.1. Файлы с точки зрения пользователя

Системы управления данными на внешних носителях появились одновременно с самими носителями, однако первоначально каждая прикладная программа сама решала проблему организации структурного хранения информации. Такая ситуация затрудняла организацию одновременного хранения на носителе данных нескольких программ и явилась предпосылкой появления централизованных систем управления внешней памятью. С технической стороны этому способствовало появление устройств, поддерживающих быстрый доступ по произвольному адресу (магнитные диски сменили магнитные ленты).

Использование централизованной системы управления внешней памятью подразумевает наличие общепринятой модели представления данных. Основным элементом этой модели является понятие файла.

Файл – именованный блок данных. Современные системы, ориентированные на работу с пользователем, оперируют понятиями файл и *файловая система* – логическая структура, организованная на базе последовательно адресуемого адресного пространства на внешнем устройстве и обеспечивающая уникальное наименование файлов и доступ к данным файлов по их именам. Для файловой системы определено множество операций, и, естественно, операции записи или чтения файла концептуально проще, чем низкоуровневые операции работы с устройствами.

Система управления файлами – это часть операционной системы, назначение которой состоит в том, чтобы организовать эффективную работу с данными, хранящимися во внешней памяти и обеспечить пользователю удобный интерфейс при работе с этими данными. Система управления файлами берет на себя распределение внешней памяти, отображение имен файлов и других объектов файловой системы в адреса внешней памяти и обеспечение доступа к данным.

Например, операционная система может разделить внешнюю память на блоки фиксированного размера (4096 байт). С точки зрения пользователя, каждый файл состоит из набора индивидуальных элементов, называемых записями (например, характеристика какого-нибудь объекта). Каждый файл хранится в виде определенной последовательности блоков (не обязательно смежных); каждый блок

хранит целое число записей. Адреса блоков, содержащих данные файла, могут быть организованы в связный список и вынесены в отдельную таблицу в памяти. Или адреса блоков данных файла могут храниться в виде массива в отдельном блоке внешней памяти (так называемом индексе или индексном узле). Или информация о расположении блоков файла на внешнем носителе может быть организована любым другим способом. К сказанному необходимо добавить, что в современных условиях файлы обычно представляют собой неструктурированную явно последовательность байтов, т.е. длина записи равна 1.

При работе с файлами операционная система хранит в оперативной памяти некоторую информацию о файлах, например, расположение файла на внешнем носителе, права доступа к файлу и т.д. В этом случае говорят о системной таблице индексов или индексных дескрипторов файлов.

Фактическим стандартом логической организации хранилища файлов является использование иерархической структуры, называемой *деревом каталогов (директорий)*. *Каталог (директория)* – специальный тип объекта файловой системы, который не хранит данные, но может содержать другие объекты файловой системы, например, файлы и другие каталоги. Файловая система имеет единственную корневую директорию, и при установлении правила, что два объекта в одной директории не могут иметь одинаковые имена, каждому файлу можно поставить в соответствие так называемое полное имя, содержащее названия всех каталогов на пути из корневой директории к данному файлу. Данное имя, во-первых, уникально, во-вторых, позволяет быстро найти файл в файловой системе.

Итак, понятие «файловая система» включает:

- совокупность всех файлов на диске;
- наборы структур данных, используемых для управления файлами, такие, например, как каталоги, дескрипторы файлов, таблицы распределения свободного и занятого пространства на диске и т.д.;
- комплекс системных программных средств, реализующих управление файлами, в частности: создание, уничтожение, чтение, запись, именование, поиск и другие операции над файлами.

Файлы управляются операционной системой. То, как они структурированы, поименованы, используются, защищены, реализованы – одна из главных тем проектирования ОС.

Основные функции файловой системы:

- 1) идентификация файлов – связывание имени файла выделенным ему пространством внешней памяти;
- 2) распределение внешней памяти между файлами;
- 3) поддержка операций управления объектами файловой системы;
- 4) обеспечение надежности и отказоустойчивости – стоимость информации может во много раз превышать стоимость компьютера;
- 5) обеспечение защиты от несанкционированного доступа;
- 6) обеспечение совместного доступа к файлам, не требуя от пользователя специальных усилий по обеспечению синхронизации доступа;
- 7) обеспечение высокой производительности.

4.2. Типы объектов файловой системы

Первые файловые системы оперировали только с одним типом объектов – файлами. Логическая структура представляла собой список файлов с различными именами. Появление понятий каталог и дерево каталогов обеспечило реализацию удобного механизма уникального именования файлов и использования при хранении иерархической структуры. Данные свойства позволяют размещать в файловой системе имена (и, возможно, содержимое) объектов, отличных от файлов и директорий. Примерами таких объектов являются:

- мягкие ссылки (ярлыки) – объекты, при выполнении над которыми определенных операций производится перенаправление операции на другой объект файловой системы;
- файлы устройств – при поступлении запроса на выполнение операции с таким объектом, вызывается функция драйвера некоторого устройства, связанного с данным объектом (связь устанавливается при создании объекта);
- файлы межпроцессного взаимодействия – при организации межпроцессного взаимодействия операционная система в качестве идентификатора системного объекта (например, мьютекса или канала) может использовать имя объекта файловой системы;
- выполнение операций над данным объектом обслуживает непосредственно подсистема организации межпроцессного взаимодействия в соответствии с логикой используемого объекта.

Существуют и другие типы объектов файловой системы, однако в п.п. 4.3 – 4.5 будут рассмотрены только два – файлы и директории.

4.3. Имена объектов файловой системы

Файлы – абстрактные объекты. Они предоставляют пользователям возможность сохранять информацию, скрывая от него детали того, как и где она хранится и то, как диски в действительности работают. Вероятно, одна из наиболее важных характеристик любого абстрактного механизма – способ наименования объектов, которыми он управляет. Когда процесс создает файл, он дает файлу имя. После завершения процесса файл продолжает существовать и через свое имя может быть доступен другим процессам.

Обычно ОС накладывают некоторые ограничения:

- на используемые в имени символы;
- длину имени;
- длину полного имени.

Например, в ОС Unix учитывается регистр символов в имени файла (case sensitive), а в MS-DOS – нет. В популярной файловой системе FAT длина имен ограничивается известной схемой 8.3 (восемь символов – собственно имя, три символа – расширение имени). Современные файловые системы, как правило, поддерживают более удобные для пользователя длинные символьные имена файлов.

4.4. Операции над файлами

Операционная система должна предоставить в распоряжение пользователя набор операций для работы с файлами, реализованных через системные вызовы. Чаще всего при выполнении пользователем одной операции ОС выполняет несколько действий. Например, находит данные файла и его атрибуты по его символьному имени; затем считывает необходимые атрибуты файла в отведенную область оперативной памяти и анализирует права пользователя на выполнение требуемой операции, и, наконец, выполняет операцию, после чего освобождает занимаемую данными файла область памяти.

Основные операции над файлами:

- Create. Создание файла, не содержащего данных. Смысл данного вызова – объявить, что файл существует и присвоить ему определенные атрибуты.
- Delete. Удаление файла и освобождение занятого им дискового пространства.

– Open. Перед использованием файла процесс должен его открыть. Цель данного системного вызова – разрешить системе проанализировать атрибуты файла и проверить права доступа к файлу, считать в оперативную память список адресов блоков файла для быстрого доступа к его данным и организовать структуры для поддержки выполнения процессом операций над файлом.

– Close. При завершении работы с файлом, его атрибуты и адреса блоков на диске больше не нужны. В этом случае файл нужно закрыть, чтобы освободить место во внутренних таблицах системы управления файлами.

– Seek. Дает возможность специфицировать место внутри файла, откуда будет производиться следующее считывание (или запись) данных, т.е. задать текущую позицию.

– Read. Чтение данных из файла. Обычно это происходит с текущей позиции. Процесс должен задать объем считываемых данных и предоставить буфер для них.

– Write. Запись данных в файл с текущей позиции. Если текущая позиция находится в конце файла, его размер увеличивается, в противном случае запись осуществляется на место имеющихся данных, которые таким образом теряются.

– Get attributes. Предоставляет процессам нужные им сведения об атрибутах файла. В качестве примера можно привести, утилиту `ls`, которая использует информацию о времени последней модификации файлов.

– Set attributes. Дает возможность процессу установить некоторые атрибуты. Наиболее очевидный пример – установка режима доступа к файлу.

– Rename. Возможность переименования файла создает дополнительные удобства для пользователя. Данная операция может быть смоделирована копированием данного файла в файл с новым именем и последующим удалением оригинала.

4.5. Директории – логическая структура файлового архива

Количество файлов на компьютере может быть большим. Многие системы хранят сотни тысяч файлов, занимающие многие гигабайты дискового пространства. Эффективное управление этими данными подразумевает наличие в них четкой логической структуры. Все современные файловые системы поддерживают многоуровневое